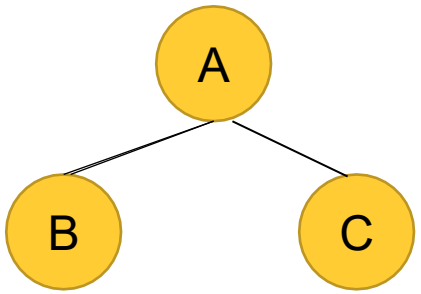
COMP 511

Computer Programming Foundations II

Binary Search Tree

Recall from last class…



PreOrder Traversal

Visit the parent first and then left and right children (PLR) A B C

InOrder Traversal

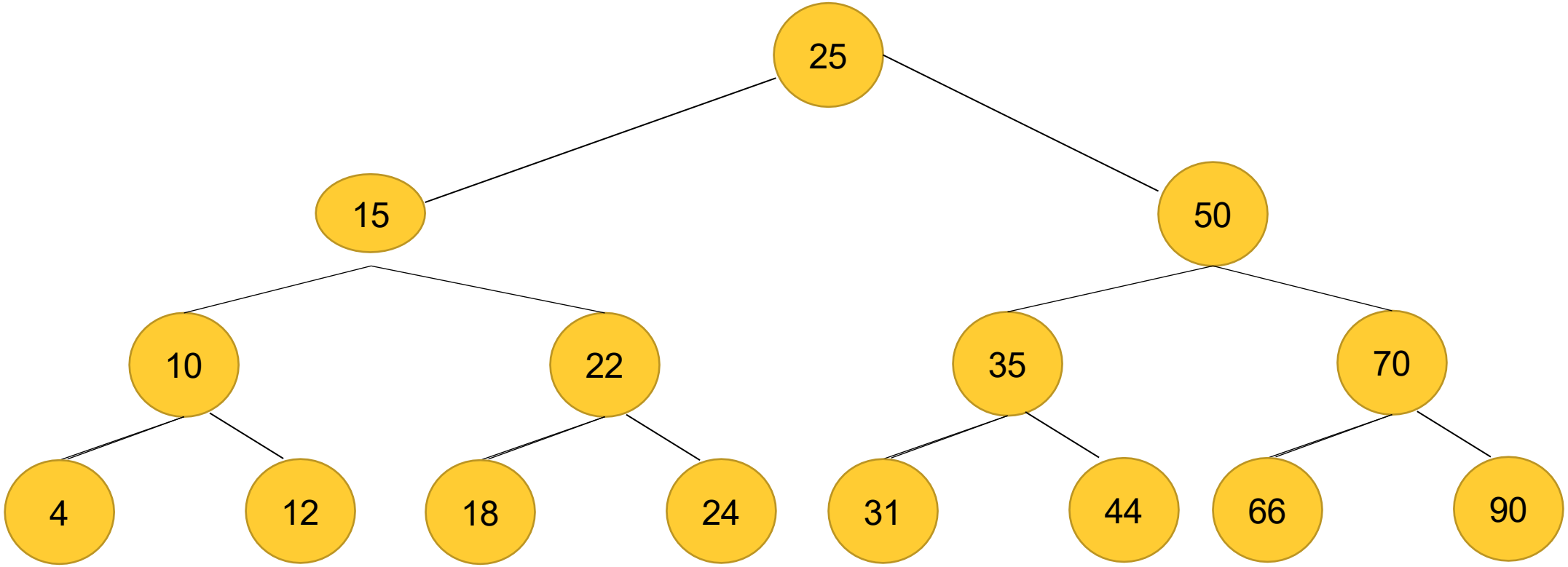
Visit the left child, then the parent and the right child (LPR) B A C

PostOrder Traversal

Visit left child, then the right child and then the parent (LRP) B C A

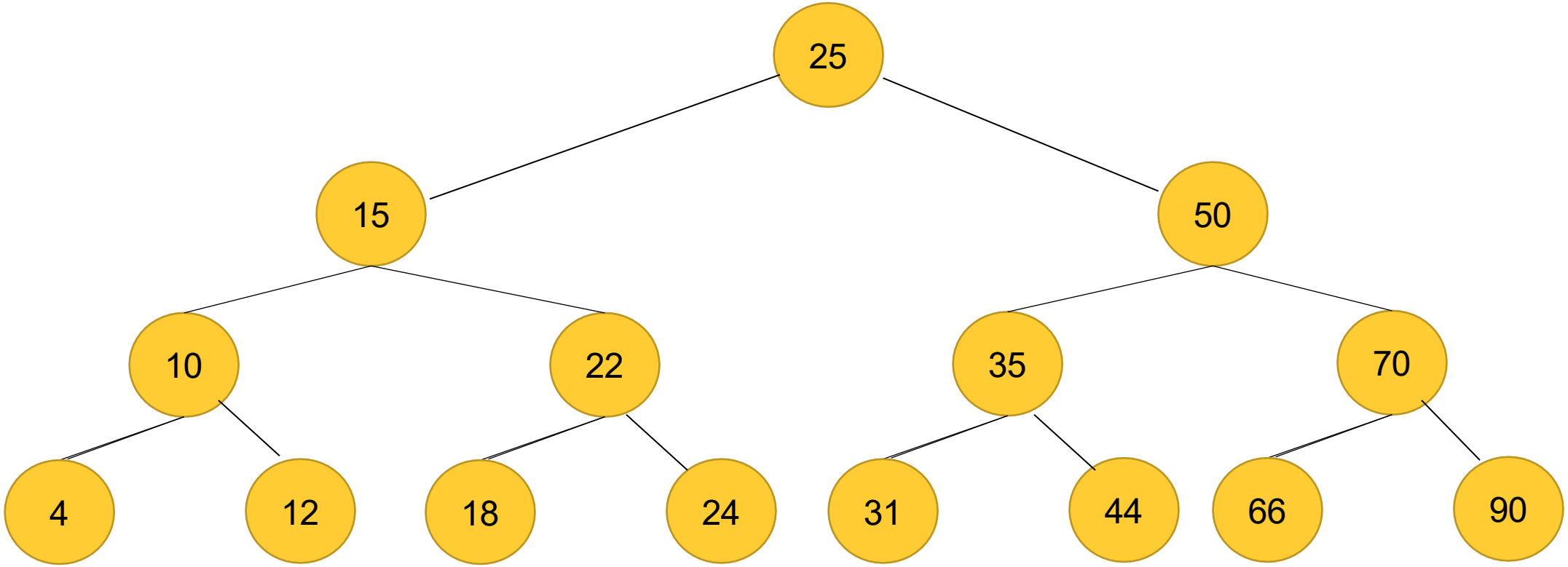
InOrder Traversal

4 10 12 15 18 22 24 25 31 35 44 50 66 70 90



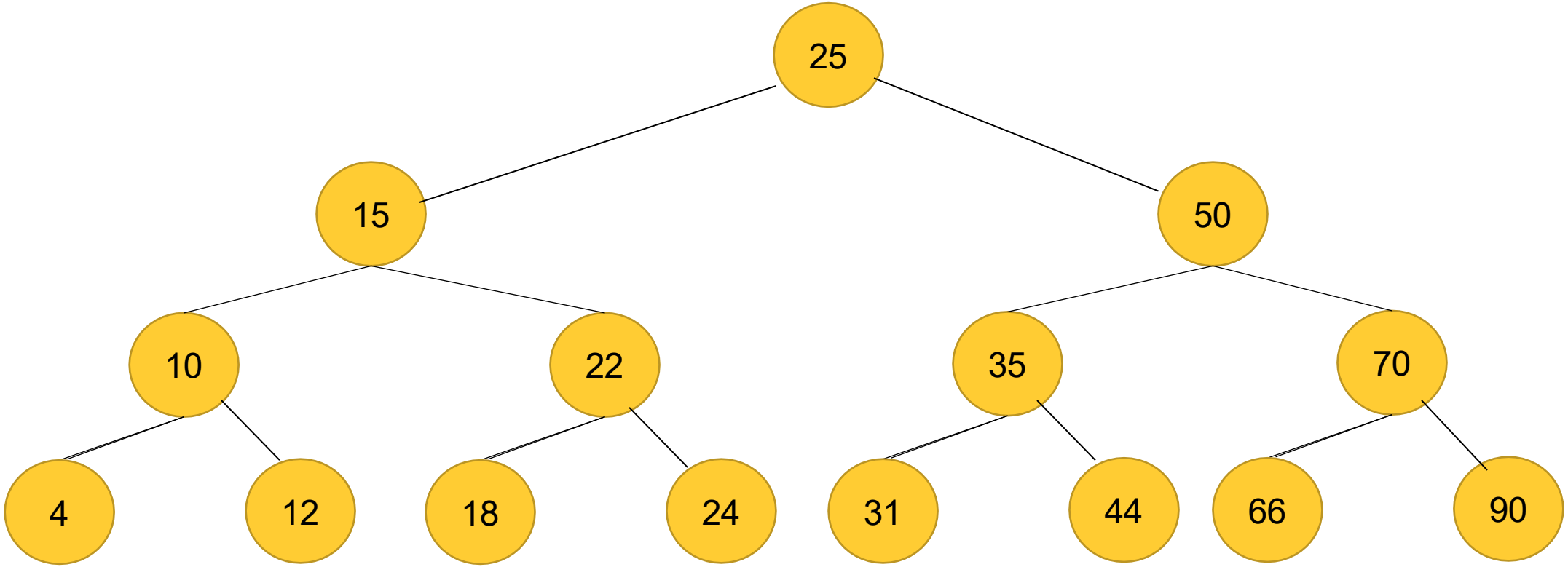
PreOrder Traversal

25 15 10 4 12 22 18 24 50 35 31 44 70 66 90



PostOrder Traversal

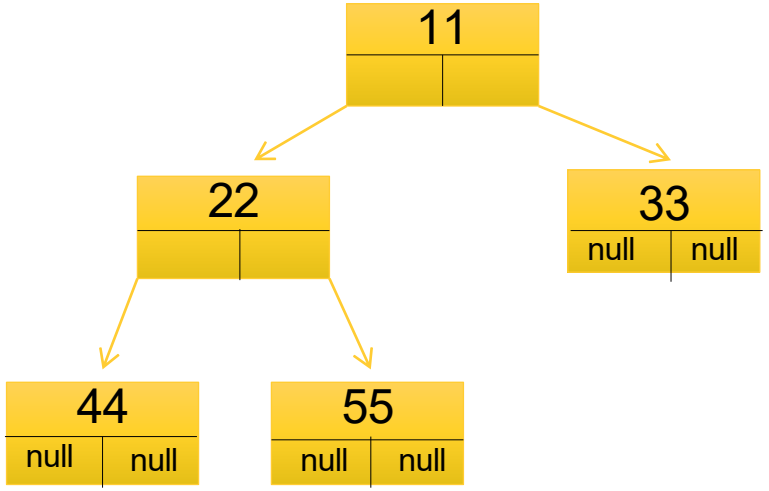
4 12 10 18 24 22 15 31 44 35 66 90 70 50 25



Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```

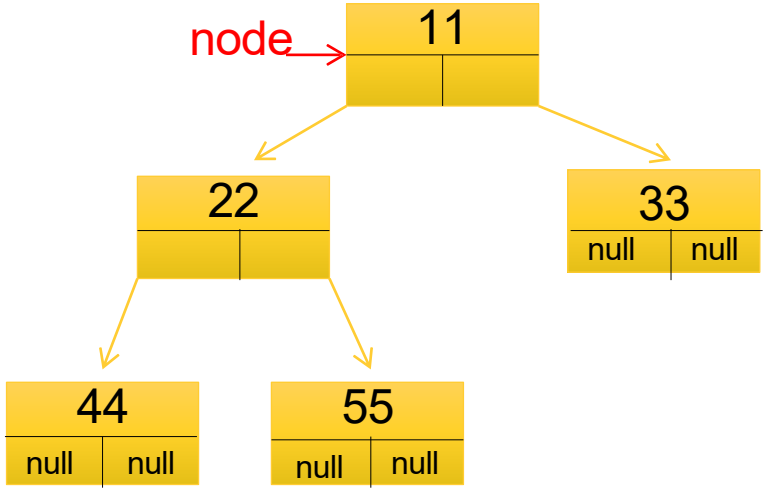


Inorder traversal:

Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



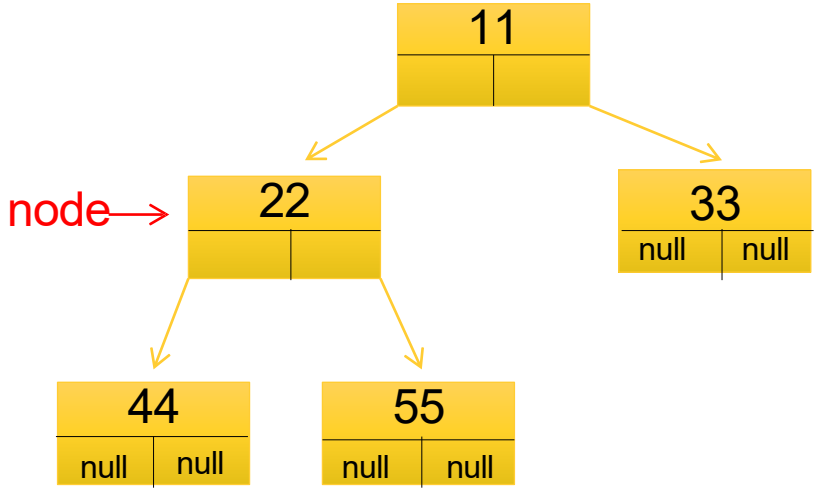
Inorder traversal:

Stack: (11)

Traversal Implementation in Java

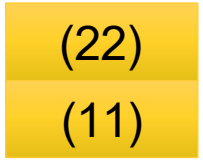
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal:

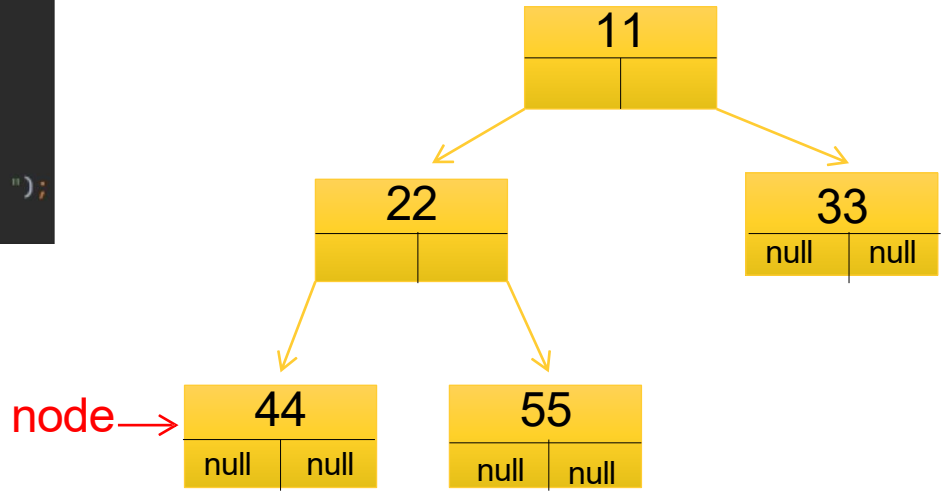
Stack:



Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



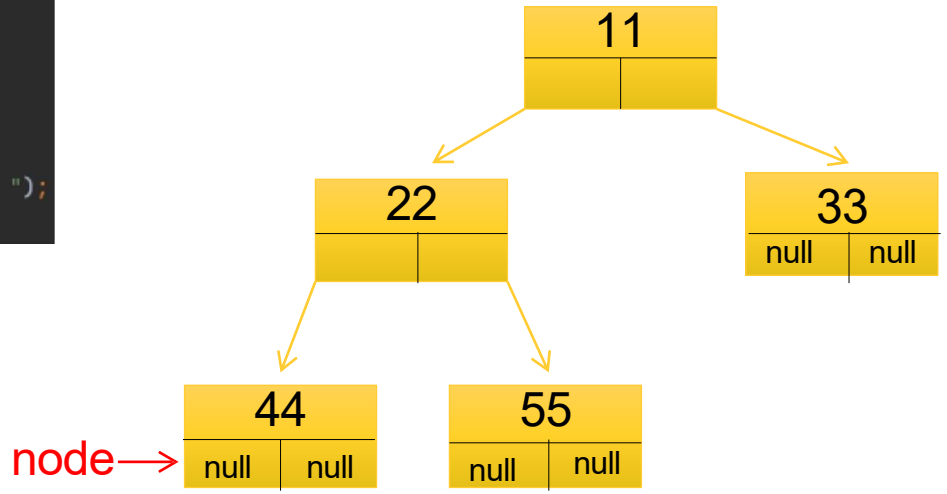
Inorder traversal:

Stack:

Traversal Implementation in Java

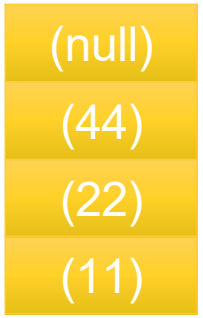
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal:

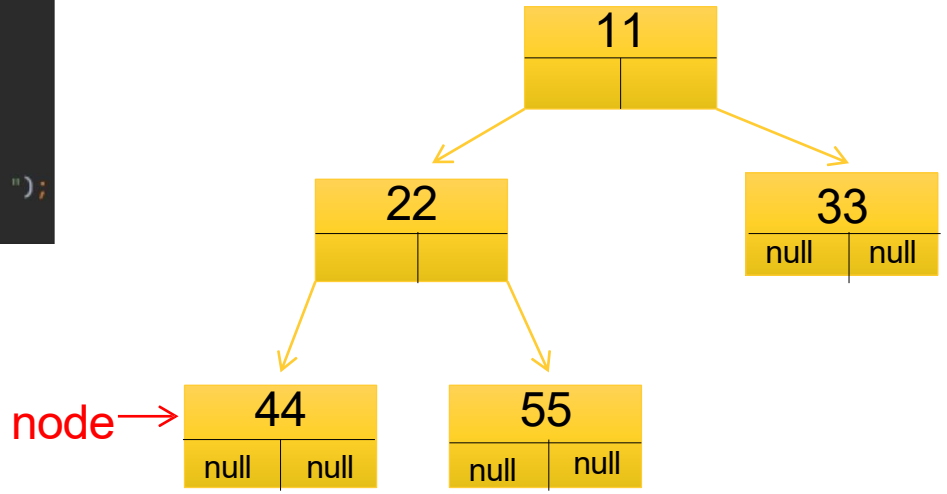
Stack:



Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal:

Stack:

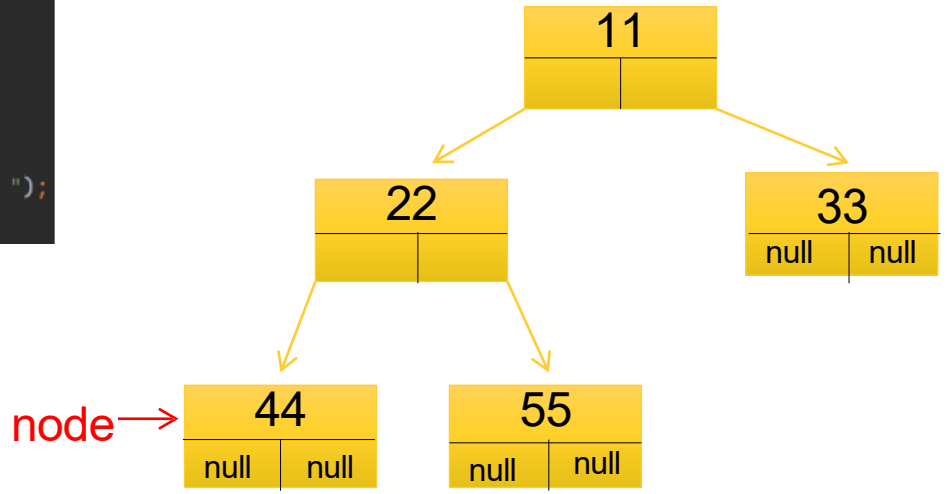
~~(null)~~
(44)
(22)
(11)

popped

Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44

Stack:

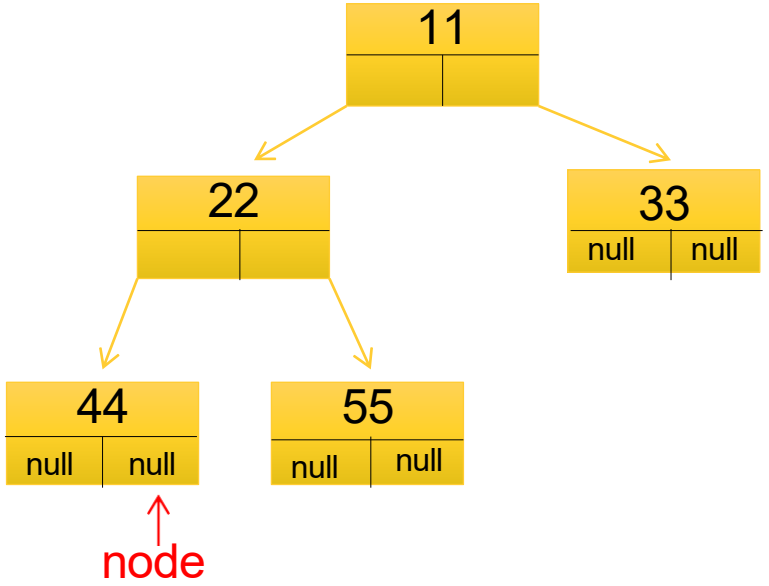
~~(null)~~
(44)
(22)
(11)

popped

Traversal Implementation in Java

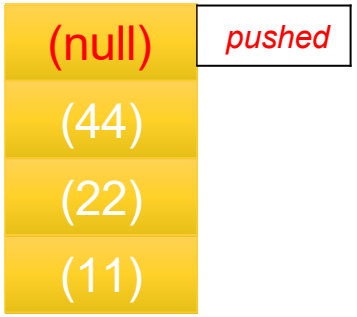
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44

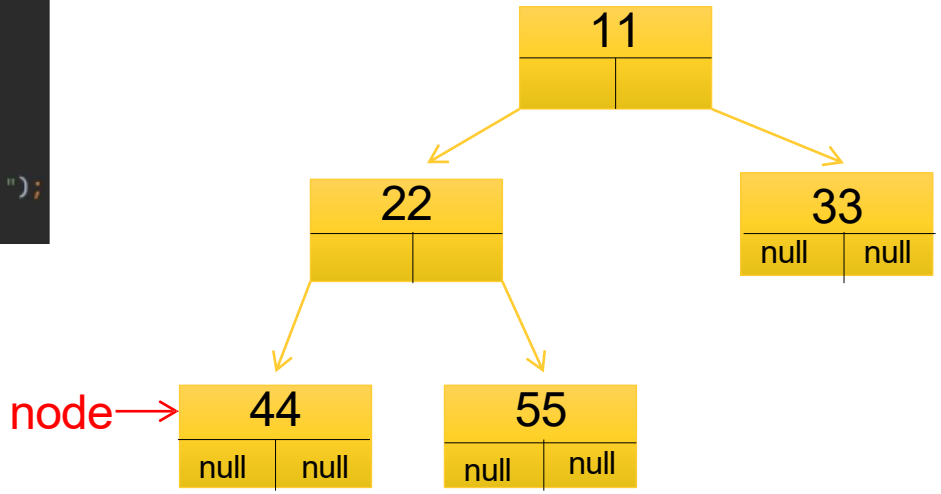
Stack:



Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44

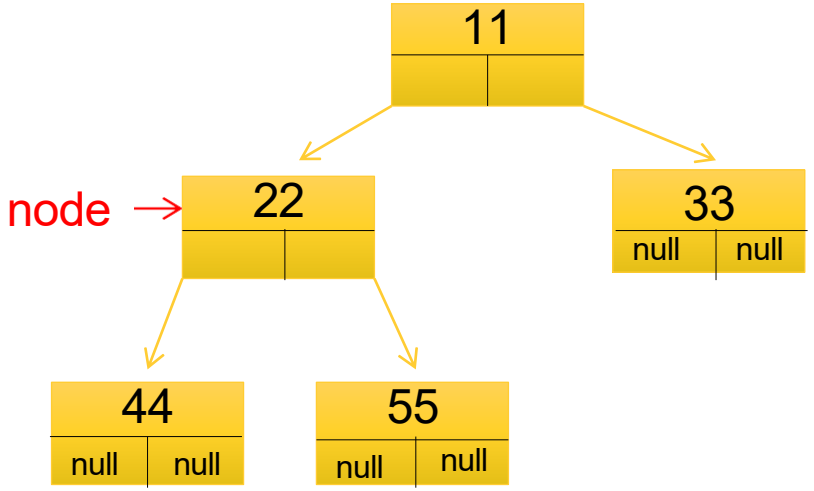
Stack:

~~(null)~~	*popped*
(44)	
(22)	
(11)	

Traversal Implementation in Java

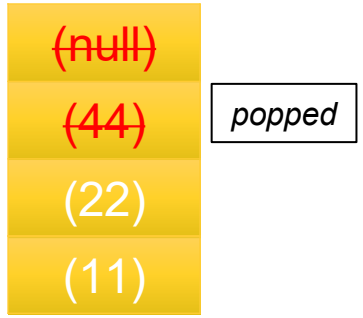
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44

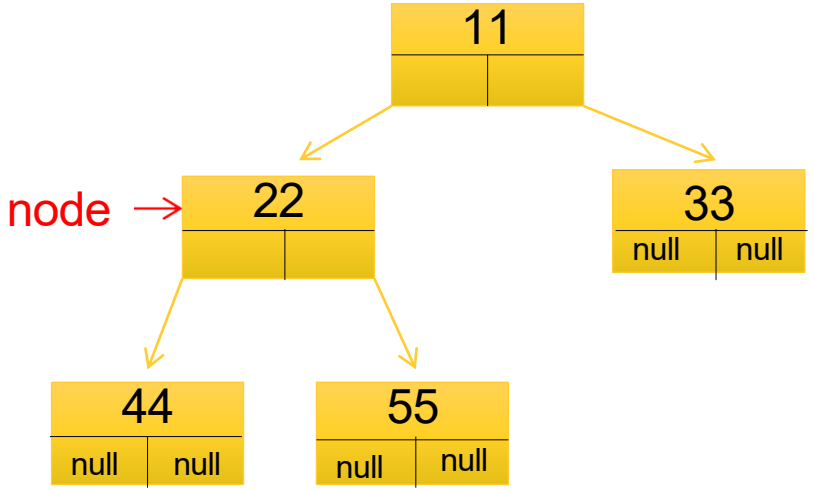
Stack:



Traversal Implementation in Java

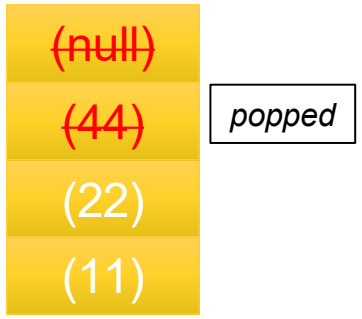
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22

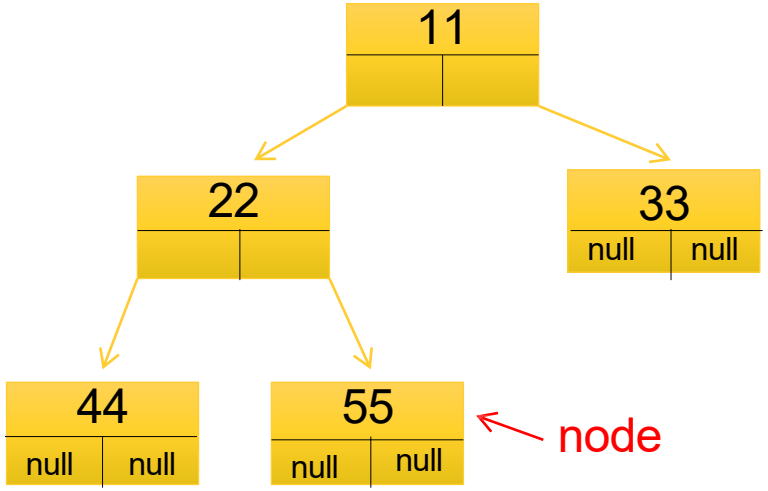
Stack:



Traversal Implementation in Java

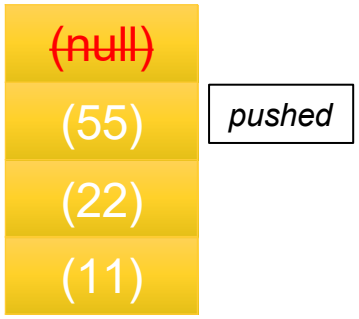
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22

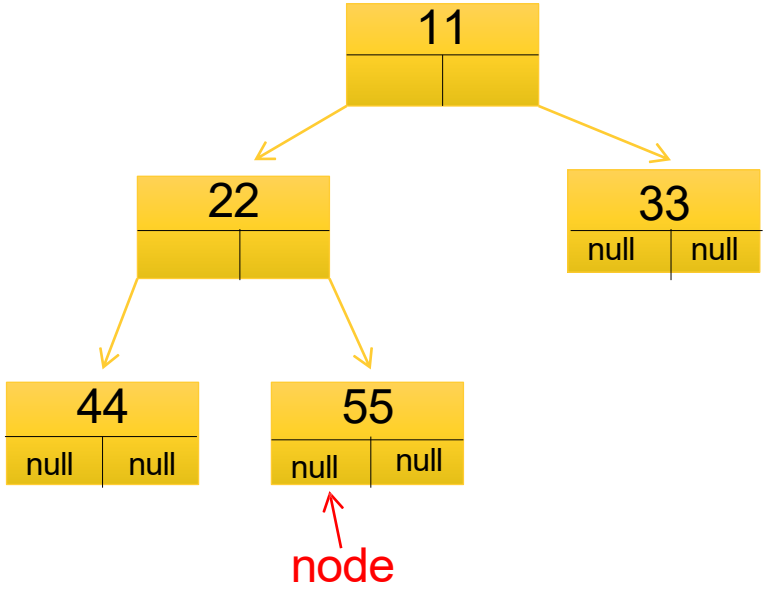
Stack:



Traversal Implementation in Java

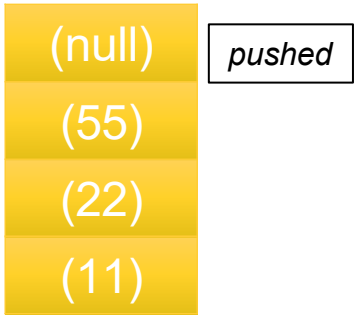
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22

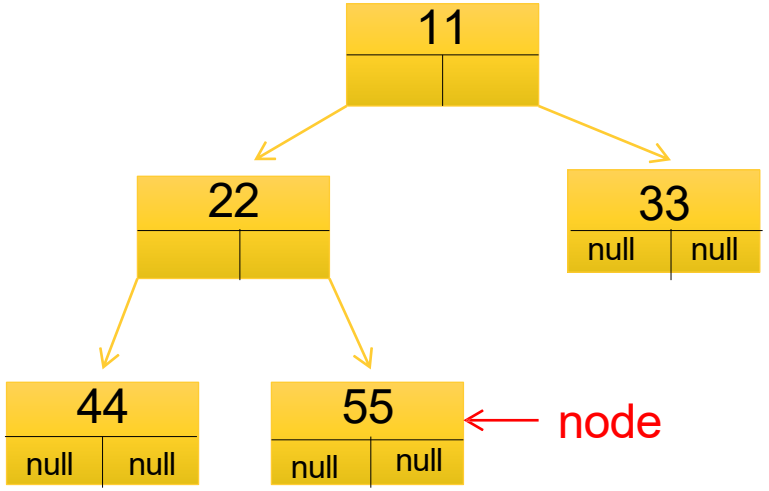
Stack:



Traversal Implementation in Java

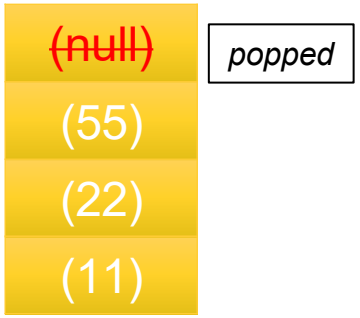
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22

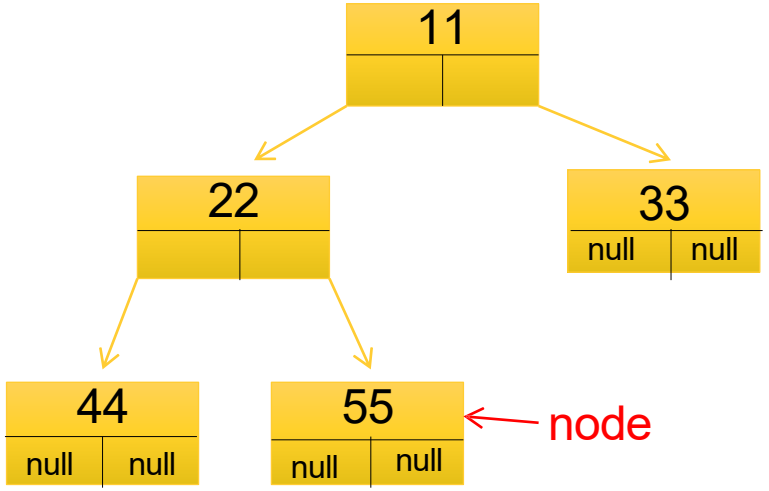
Stack:



Traversal Implementation in Java

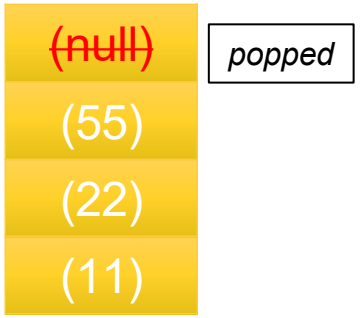
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55

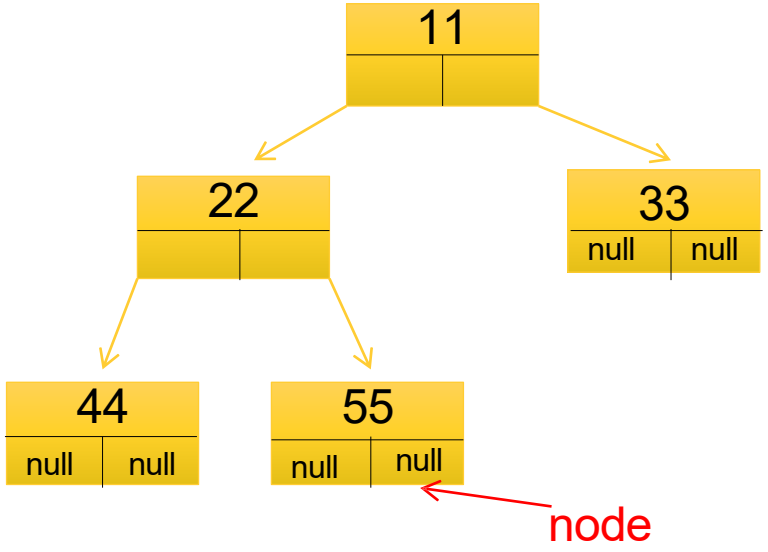
Stack:



Traversal Implementation in Java

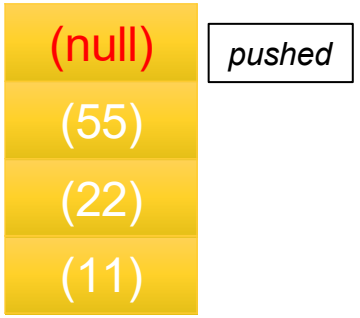
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55

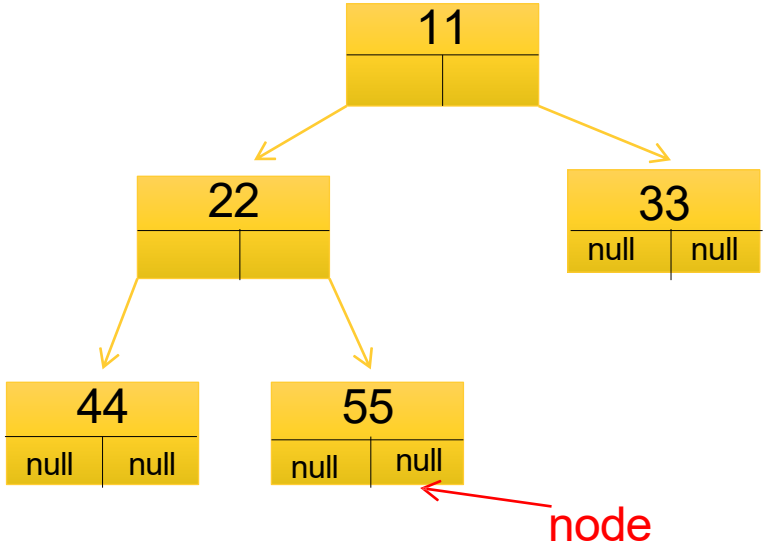
Stack:



Traversal Implementation in Java

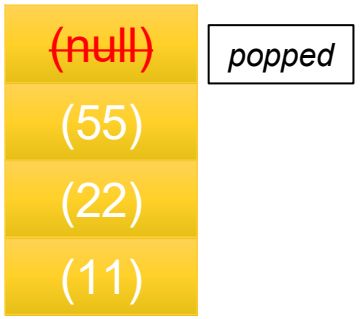
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55

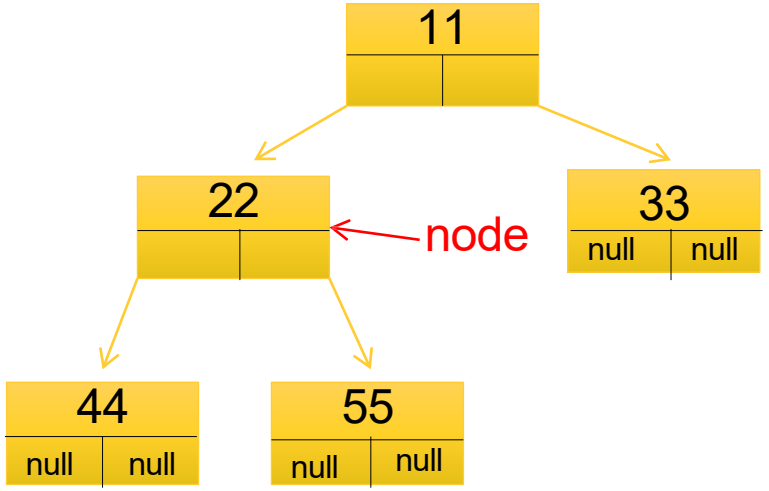
Stack:



Traversal Implementation in Java

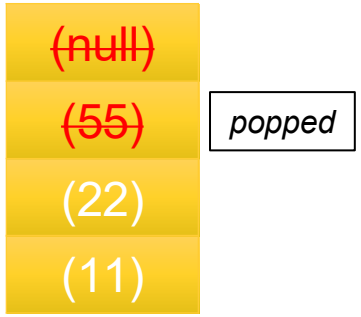
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55

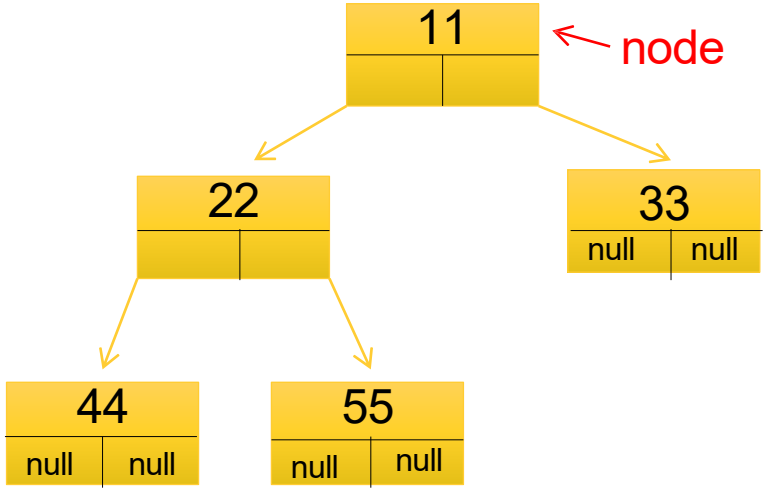
Stack:



Traversal Implementation in Java

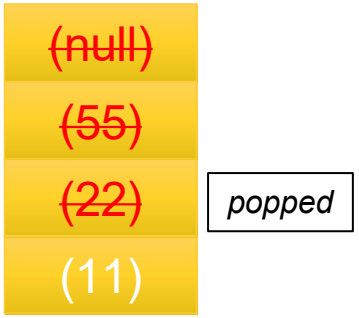
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55 11

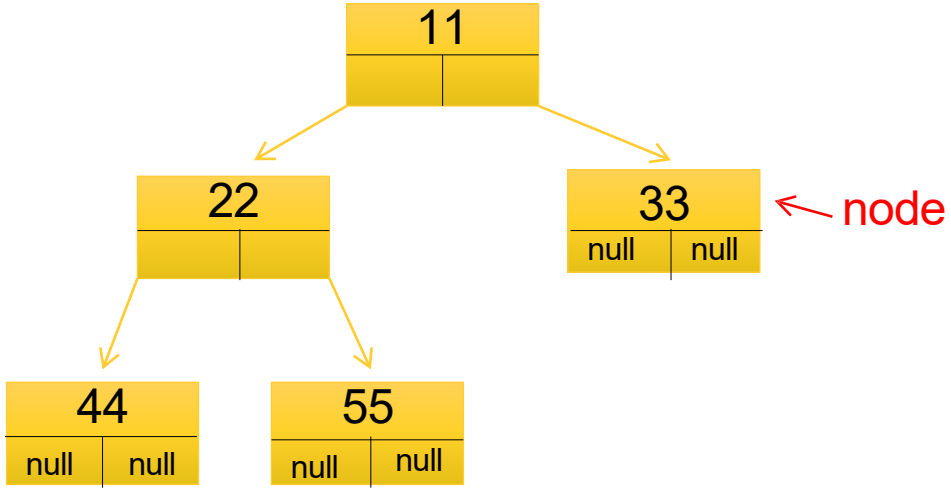
Stack:



Traversal Implementation in Java

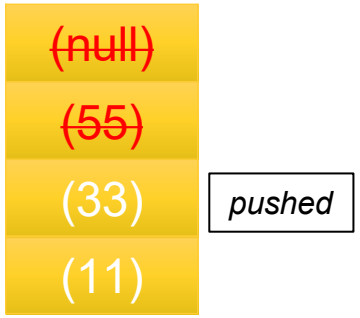
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55 11

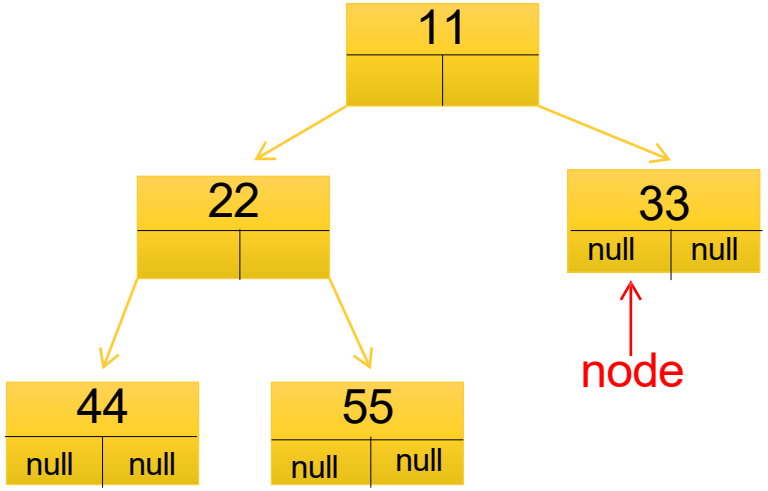
Stack:



Traversal Implementation in Java

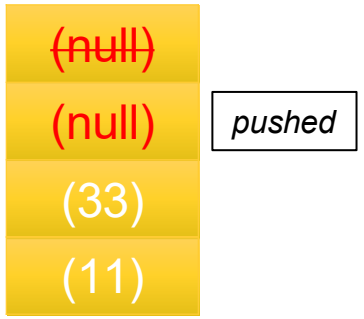
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55 11

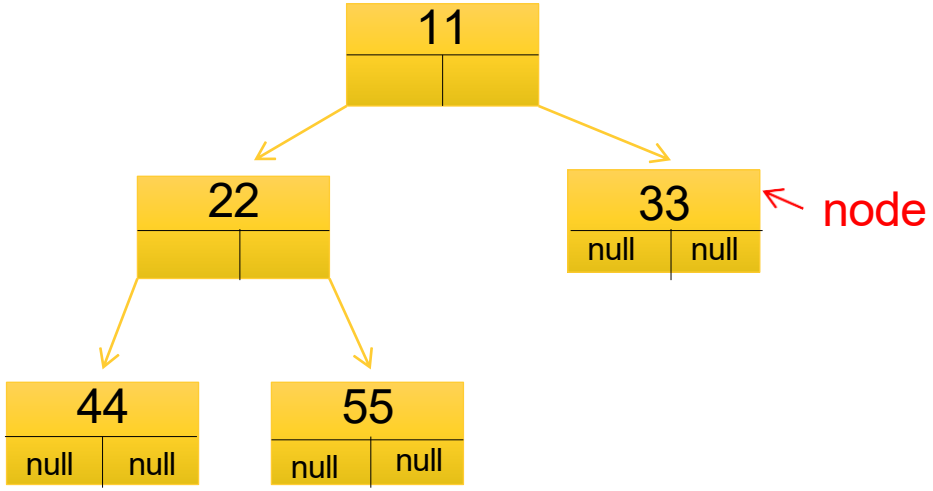
Stack:



Traversal Implementation in Java

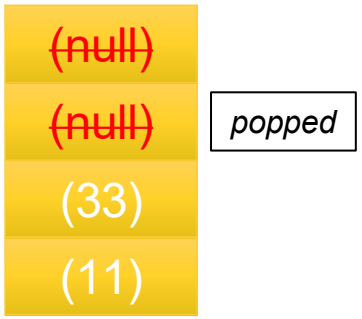
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55 11

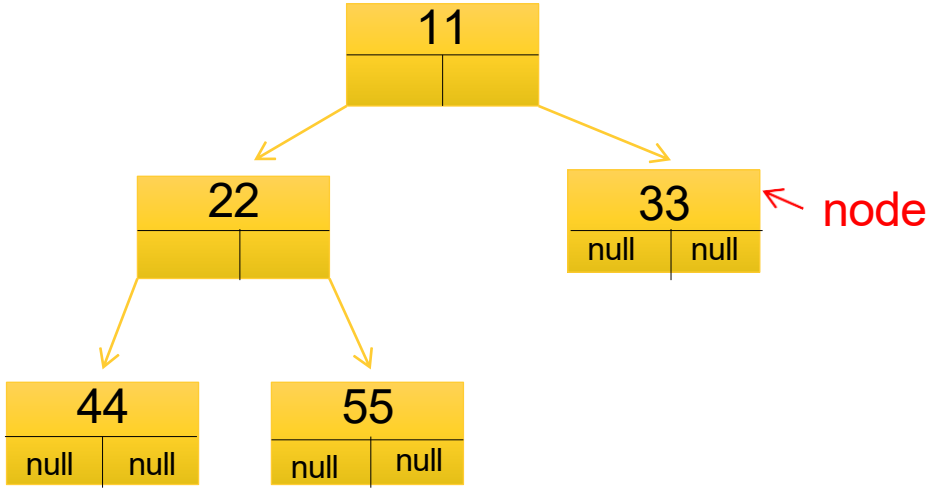
Stack:



Traversal Implementation in Java

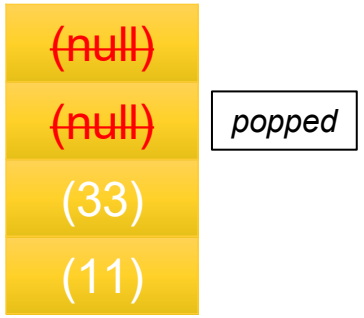
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55 11 33

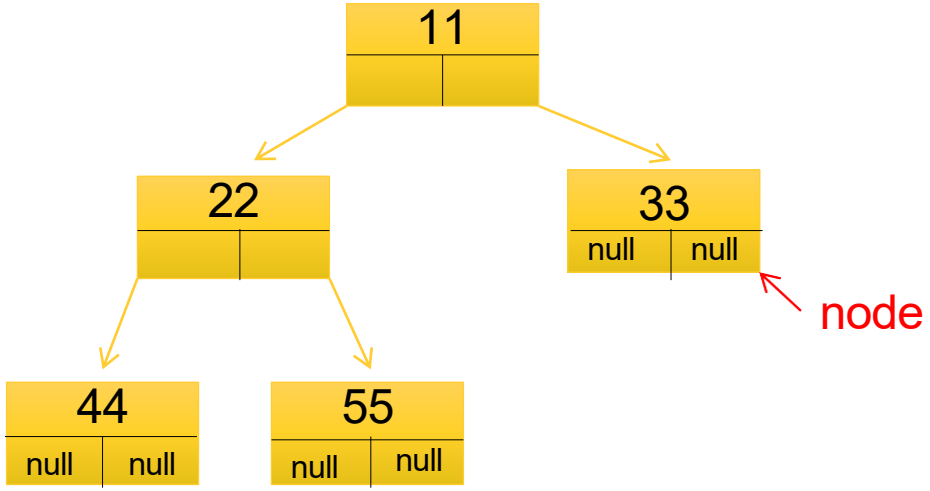
Stack:



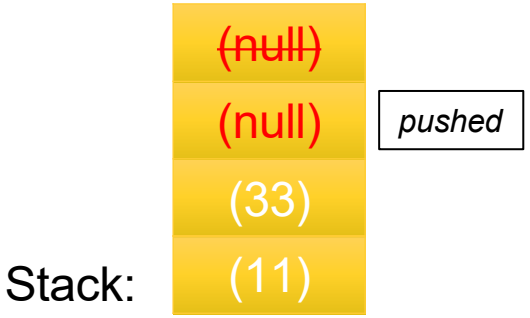
Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



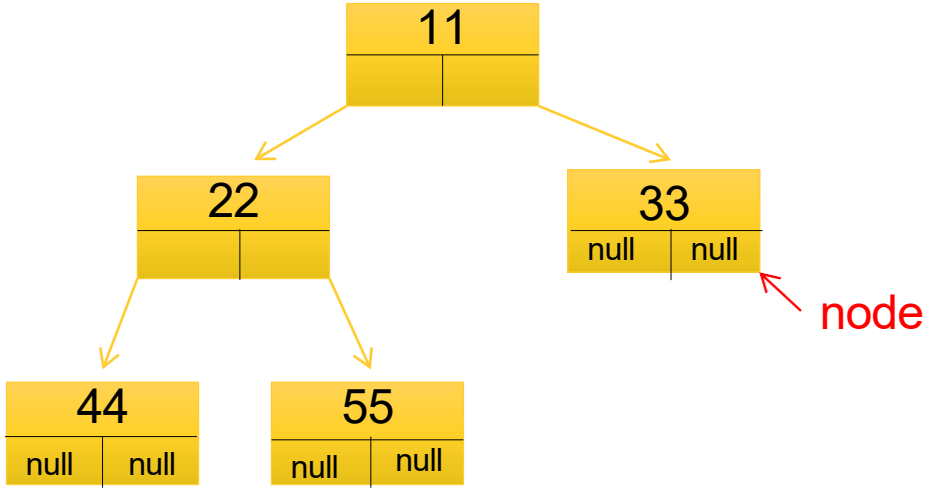
Inorder traversal: 44 22 55 11 33



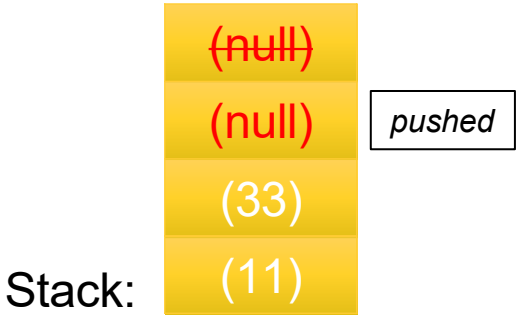
Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



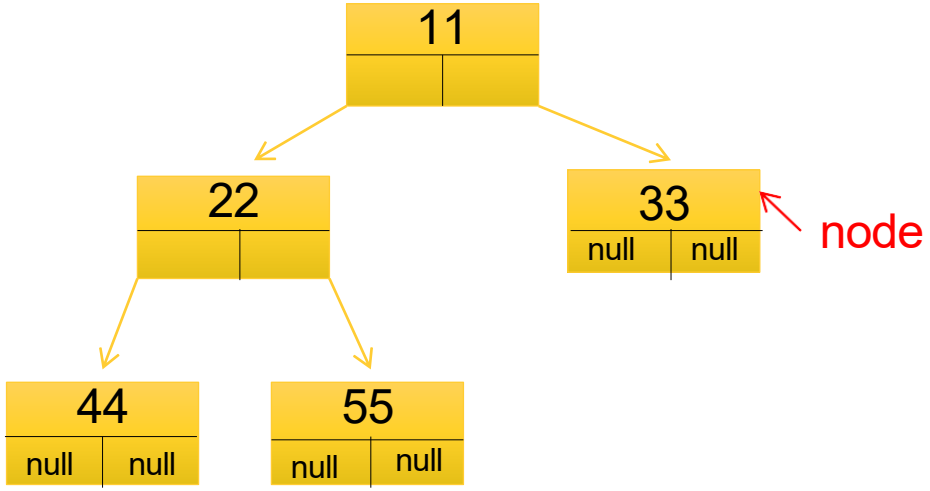
Inorder traversal: 44 22 55 11 33



Traversal Implementation in Java

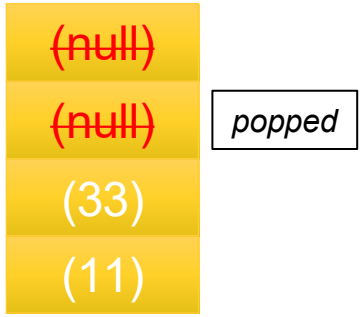
```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55 11 33

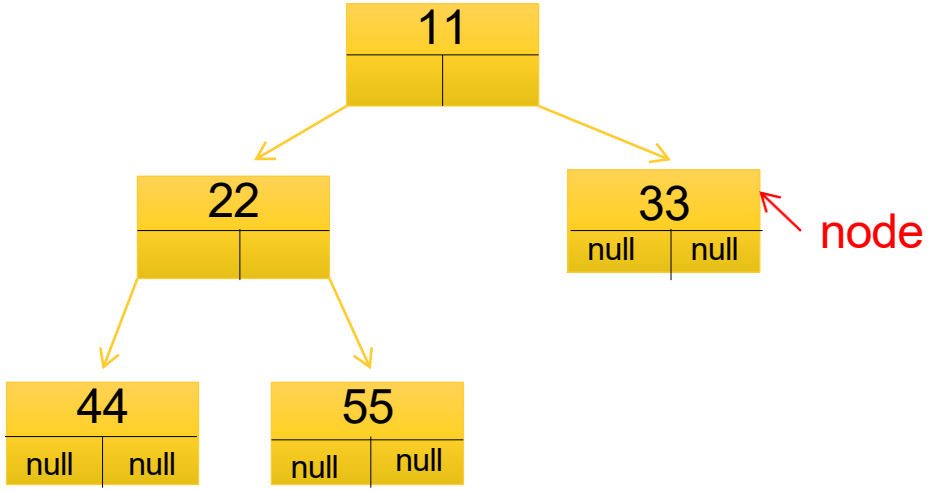
Stack:



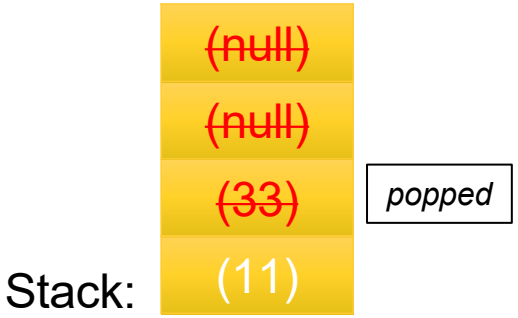
Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



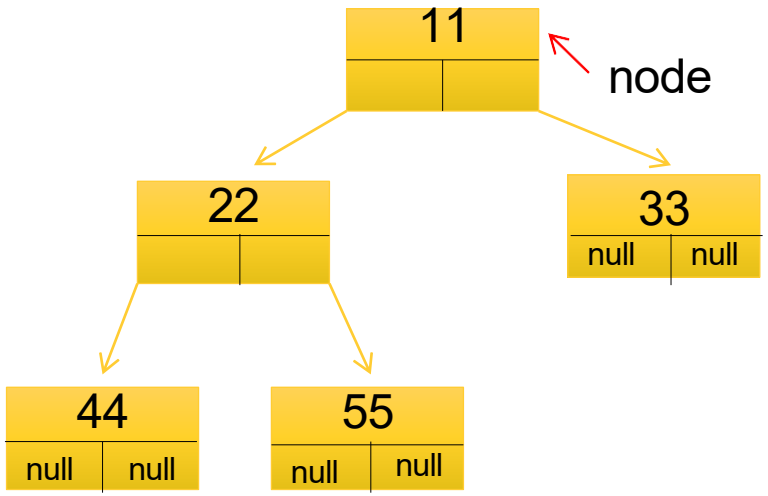
Inorder traversal: 44 22 55 11 33



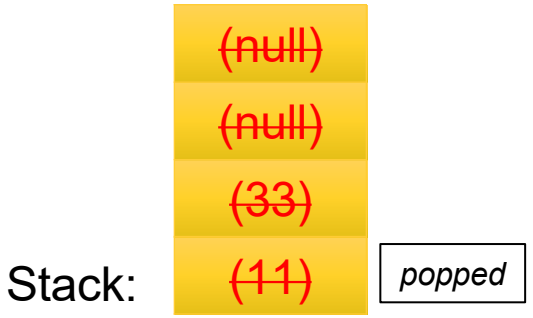
Traversal Implementation in Java

```java
    // Main method
    public static void main(String[] args)
    {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(11);
        tree.root.left = new Node(22);
        tree.root.right = new Node(33);
        tree.root.left.left = new Node(44);
        tree.root.left.right = new Node(55);

        System.out.println("\nInorder traversal of binary tree is ");
        tree.printInorder();
```

```java
    /* Print nodes according to the
       Inorder traversal. */
    void printInorder(Node node)
    {
        if (node == null)
            return;

        /* first recur on left child */
        printInorder(node.left);

        /* then print the data of node */
        System.out.print(node.key + " ");

        /* now recur on right child */
        printInorder(node.right);
    }
```



Inorder traversal: 44 22 55 11 33

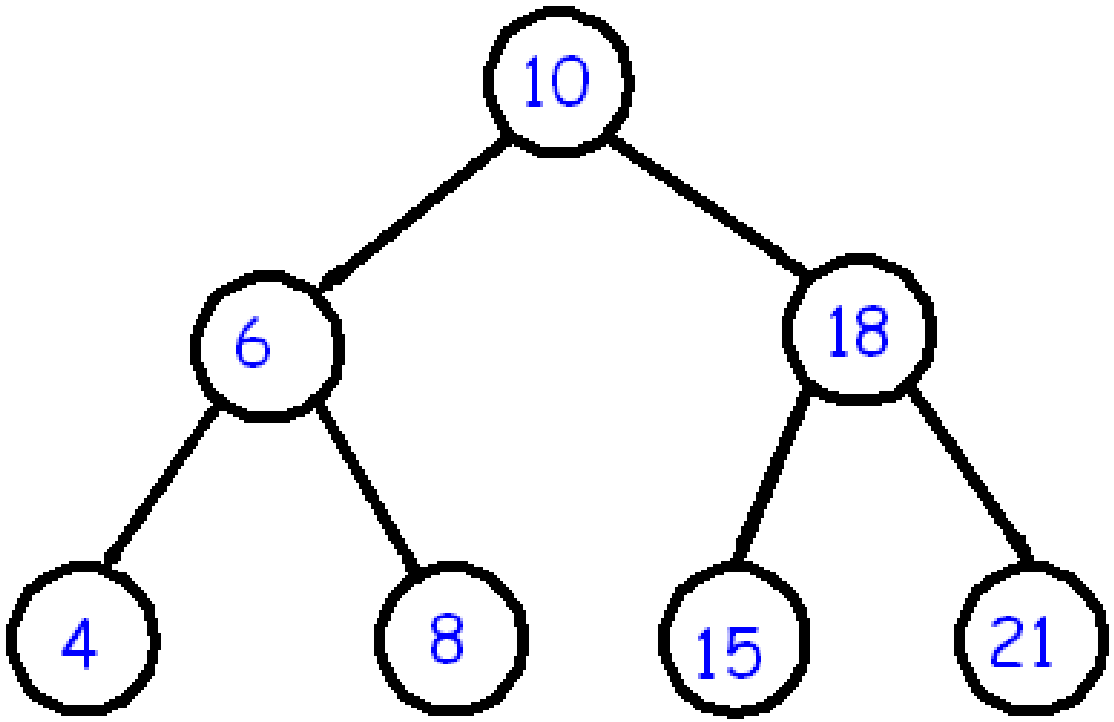


Basic operation in Binary Search Tree

Search for a number

Add a new number

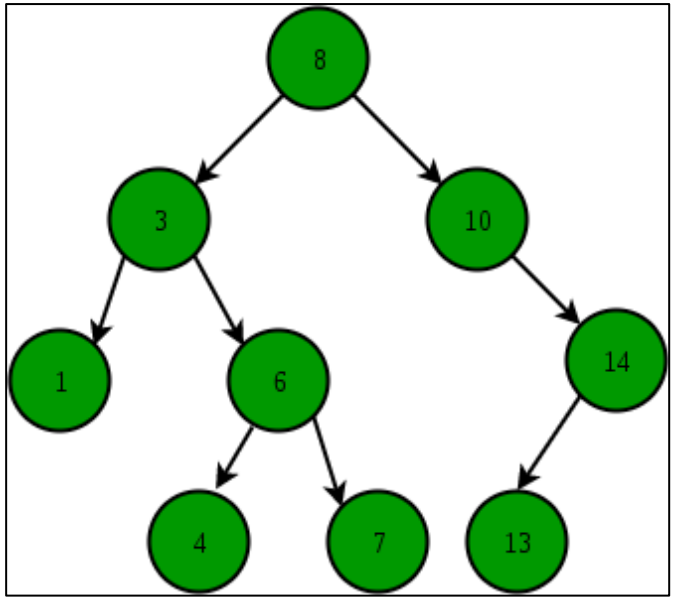
Delete a number



Searching in Binary Search Tree

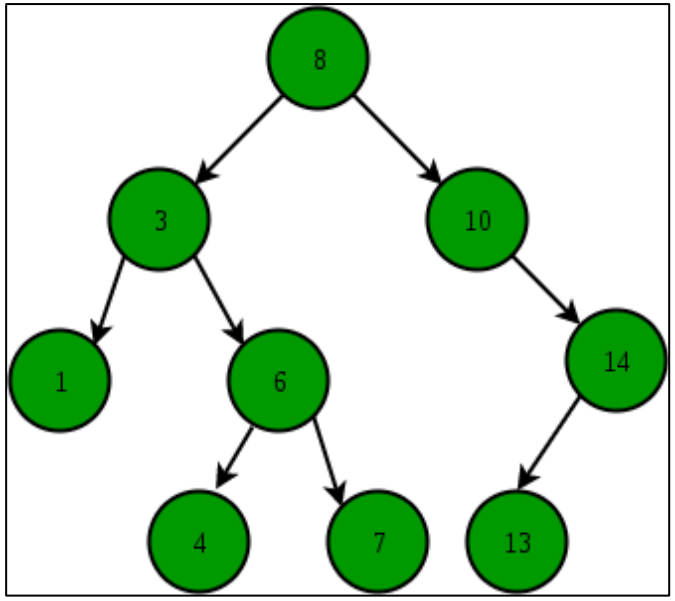
Start from the root

```java
public Node search(Node root, int key)
{
    if (root==null || root.key==key){
        return root;
    }

    // val is greater than root's key
    if (root.key > key)
        return search(root.left, key);

    // val is less than root's key
    return search(root.right, key);
}
```



Insertion in Binary Search Tree

Insert as a leave node



Insertion in Binary Search Tree

```
public static void main(String[] args)
   {
      BinarySearchTree bst = new BinarySearchTree();

      bst.insert(43);
      bst.insert(26);
      bst.insert(22);
      bst.insert(31);
      bst.insert(68);
      bst.insert(67);
      bst.insert(95);

      System.out.println("Inorder traversal:");
      bst.inorder();

      System.out.println("\nDelete 22");
      bst.deleteKey(22);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 26");
      bst.deleteKey(26);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 43");
      bst.deleteKey(43);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();
   }
}
```

```
        43
      /    \
    26      68
   /  \    /  \
22    31 67    95
```

Insertion in Binary Search Tree

bst.insert(43);

```
void insert(int key)
   {
     /* Note that root is the root of the bst.
          root is only modified in the event
          that  the bst was empty prior to this
          insertion.  */
      root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

      if (root == null)
      {
          root = new Node(key);
          return root;
      }

      if (key < root.key)
          root.left = insertRec(root.left, key);
      else if (key > root.key)
          root.right = insertRec(root.right, key);

      return root;
   }
```

Root	key
null	43

Insertion in Binary Search Tree

bst.insert(43);

```
void insert(int key)
   {
     /* Note that root is the root of the bst.
          root is only modified in the event
          that  the bst was empty prior to this
          insertion.  */
      root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

      if (root == null)
      {
          root = new Node(key);
          return root;
      }

      if (key < root.key)
          root.left = insertRec(root.left, key);
      else if (key > root.key)
          root.right = insertRec(root.right, key);

      return root;
   }
```

Insertion in Binary Search Tree

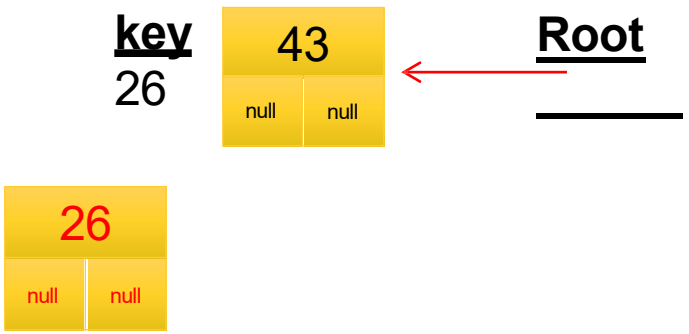
bst.insert(26);

```
void insert(int key)
   {
      /* Note that root is the root of the bst.
           root is only modified in the event
           that  the bst was empty prior to this
           insertion.  */
       root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

       if (root == null)
       {
           root = new Node(key);
           return root;
       }

       if (key < root.key)
           root.left = insertRec(root.left, key);
       else if (key > root.key)
           root.right = insertRec(root.right, key);

       return root;
   }
```

Insertion in Binary Search Tree

bst.insert(26);

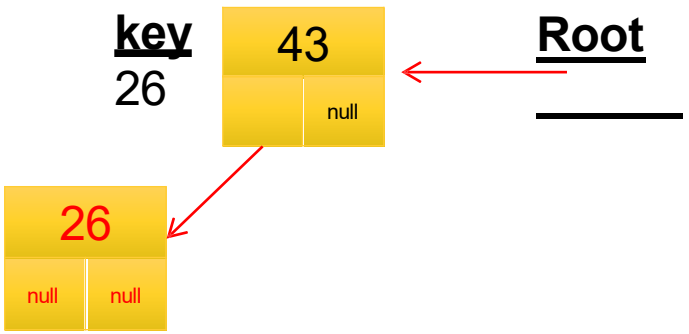
```
void insert(int key)
   {
      /* Note that root is the root of the bst.
           root is only modified in the event
           that  the bst was empty prior to this
           insertion.  */
       root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

       if (root == null)
       {
           root = new Node(key);
           return root;
       }

       if (key < root.key)
           root.left = insertRec(root.left, key);
       else if (key > root.key)
           root.right = insertRec(root.right, key);

       return root;
   }
```



Insertion in Binary Search Tree

bst.insert(26);

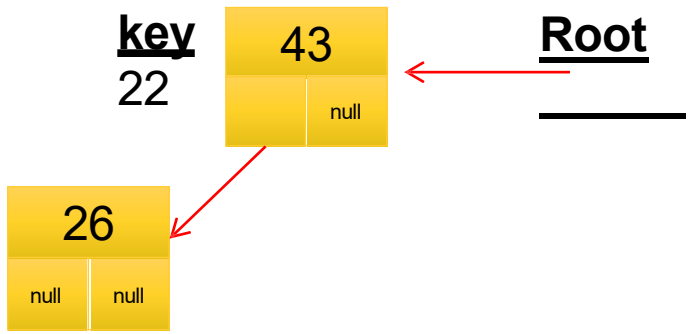
```
void insert(int key)
   {
      /* Note that root is the root of the bst.
           root is only modified in the event
           that  the bst was empty prior to this
           insertion.  */
       root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

       if (root == null)
       {
           root = new Node(key);
           return root;
       }

       if (key < root.key)
           root.left = insertRec(root.left, key);
       else if (key > root.key)
           root.right = insertRec(root.right, key);

       return root;
   }
```



Insertion in Binary Search Tree

bst.insert(22);

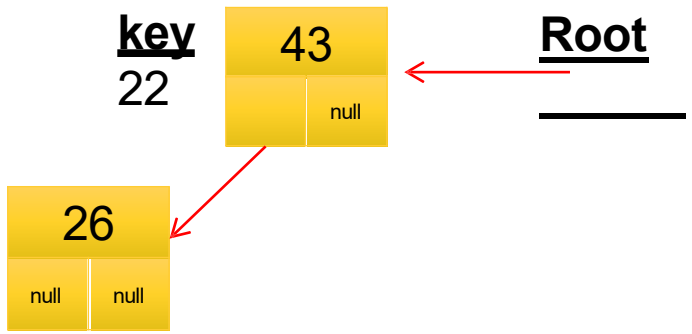
```
void insert(int key)
   {
      /* Note that root is the root of the bst.
           root is only modified in the event
           that  the bst was empty prior to this
           insertion.  */
       root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

       if (root == null)
       {
           root = new Node(key);
           return root;
       }

       if (key < root.key)
           root.left = insertRec(root.left, key);
       else if (key > root.key)
           root.right = insertRec(root.right, key);

       return root;
   }
```



Insertion in Binary Search Tree

bst.insert(22);

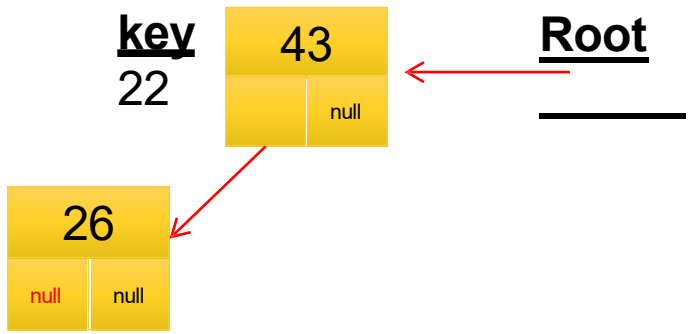
```
void insert(int key)
   {
      /* Note that root is the root of the bst.
           root is only modified in the event
           that  the bst was empty prior to this
           insertion.  */
       root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

       if (root == null)
       {
           root = new Node(key);
           return root;
       }

       if (key < root.key)
           root.left = insertRec(root.left, key);
       else if (key > root.key)
           root.right = insertRec(root.right, key);

       return root;
   }
```



Insertion in Binary Search Tree

bst.insert(22);

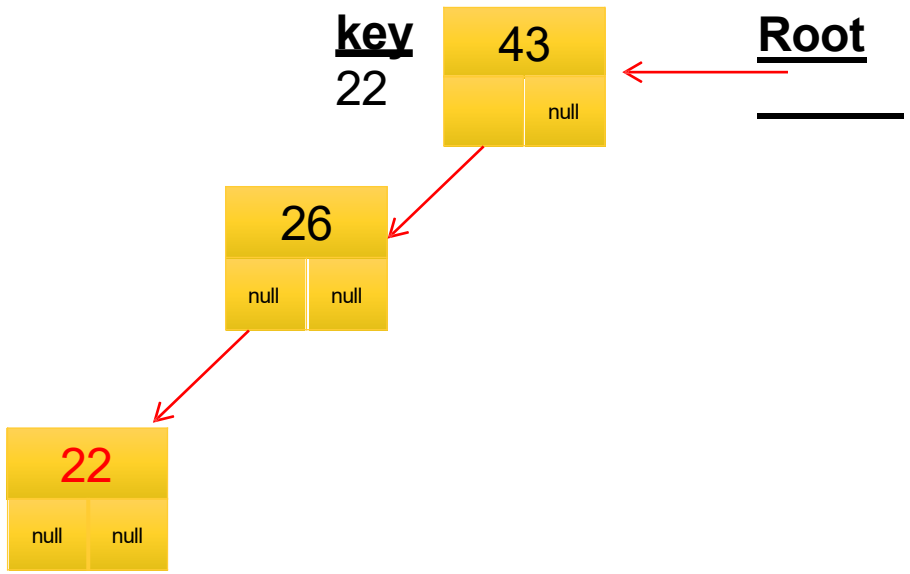
```
void insert(int key)
   {
      /* Note that root is the root of the bst.
           root is only modified in the event
           that  the bst was empty prior to this
           insertion.  */
       root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

       if (root == null)
       {
           root = new Node(key);
           return root;
       }

       if (key < root.key)
           root.left = insertRec(root.left, key);
       else if (key > root.key)
           root.right = insertRec(root.right, key);

       return root;
   }
```



Insertion in Binary Search Tree

bst.insert(22);

```
void insert(int key)
   {
      /* Note that root is the root of the bst.
           root is only modified in the event
           that  the bst was empty prior to this
           insertion.  */
       root = insertRec(root, key);
   }

   /* Note the use of recursion. */
   Node insertRec(Node root, int key)
   {

       if (root == null)
       {
           root = new Node(key);
           return root;
       }

       if (key < root.key)
           root.left = insertRec(root.left, key);
       else if (key > root.key)
           root.right = insertRec(root.right, key);

       return root;
   }
```



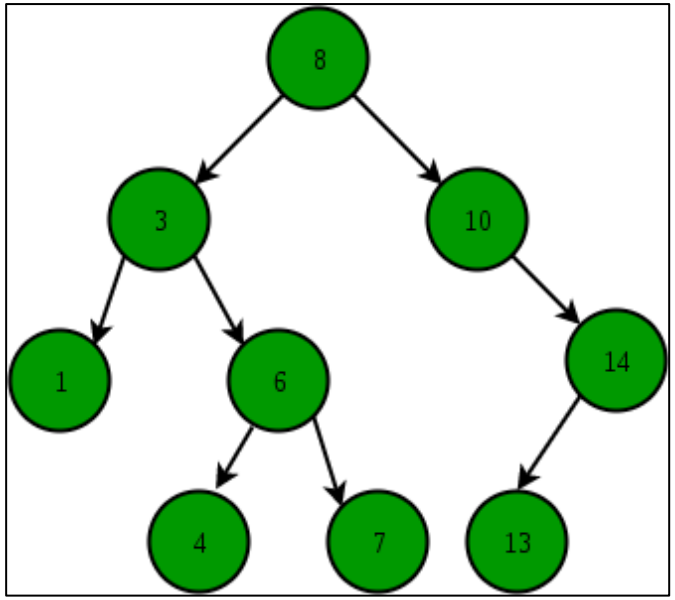
In-order Traversal

```
public static void main(String[] args)
   {
      BinarySearchTree bst = new BinarySearchTree();

      bst.insert(43);
      bst.insert(26);
      bst.insert(22);
      bst.insert(31);
      bst.insert(68);
      bst.insert(67);
      bst.insert(95);

      System.out.println("Inorder traversal:");
      bst.inorder();

      System.out.println("\nDelete 22");
      bst.deleteKey(22);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 26");
      bst.deleteKey(26);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 43");
      bst.deleteKey(43);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();
   }
}
```

```
        43
      /    \
    26      68
   /  \    /  \
22    31  67   95
```

Inorder traversal:
22 26 31 43 67 68 95

Deletion in Binary Search Tree



1. **Delete a leave node**

2. **Delete a node with one child**

3. **Delete a node with two children**

   - inorder successor

Deletion in Binary Search Tree

```
public static void main(String[] args)
   {
      BinarySearchTree bst = new BinarySearchTree();

      bst.insert(43);
      bst.insert(26);
      bst.insert(22);
      bst.insert(31);
      bst.insert(68);
      bst.insert(67);
      bst.insert(95);

      System.out.println("Inorder traversal:");
      bst.inorder();

      System.out.println("\nDelete 22");
      bst.deleteKey(22);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 26");
      bst.deleteKey(26);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 43");
      bst.deleteKey(43);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();
   }
}
```

```
        43
      /    \
    26      68
      \    /  \
      31  67   95
```

Inorder traversal:
26 31 43 67 68 95

Deletion in Binary Search Tree

```
public static void main(String[] args)
   {
      BinarySearchTree bst = new BinarySearchTree();

      bst.insert(43);
      bst.insert(26);
      bst.insert(22);
      bst.insert(31);
      bst.insert(68);
      bst.insert(67);
      bst.insert(95);

      System.out.println("Inorder traversal:");
      bst.inorder();

      System.out.println("\nDelete 22");
      bst.deleteKey(22);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 26");
      bst.deleteKey(26);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 43");
      bst.deleteKey(43);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();
   }
}
```

```
        43
      /    \
    31      68
           /  \
         67    95
```

Inorder traversal:
31 43 67 68 95

Deletion in Binary Search Tree

```
public static void main(String[] args)
   {
      BinarySearchTree bst = new BinarySearchTree();

      bst.insert(43);
      bst.insert(26);
      bst.insert(22);
      bst.insert(31);
      bst.insert(68);
      bst.insert(67);
      bst.insert(95);

      System.out.println("Inorder traversal:");
      bst.inorder();

      System.out.println("\nDelete 22");
      bst.deleteKey(22);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 26");
      bst.deleteKey(26);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();

      System.out.println("\nDelete 43");
      bst.deleteKey(43);
      System.out.println("Inorder traversal of the modified bst");
      bst.inorder();
   }
}
```

```
        43 (?)
      /      \
    31        68
             /  \
           67    95
```

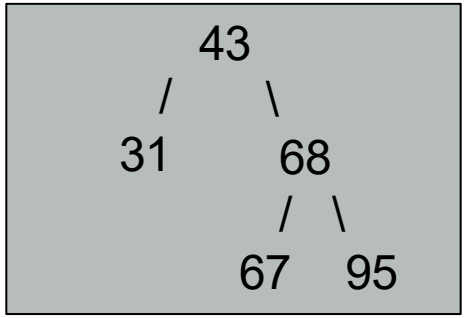
Deletion in Binary Search Tree

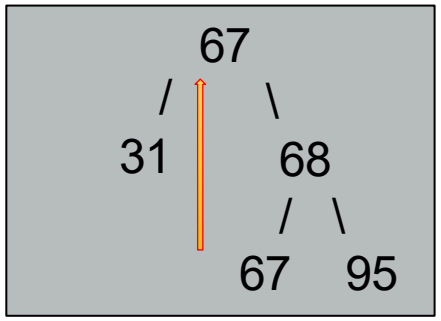
```
void deleteKey(int key)
   {
      root = deleteRec(root, key);
   }

   /* A recursive function to delete a key in BST */
   Node deleteRec(Node root, int key)
   {
      /* Base Case: If the bst is empty */
      if (root == null)  return root;

      /* Otherwise, recur down the bst */
      if (key < root.key)
         root.left = deleteRec(root.left, key);
      else if (key > root.key)
         root.right = deleteRec(root.right, key);

      // if key is same as root's key, then This is the node
      // to be deleted
      else
      {
         // node with only one child or no child
         if (root.left == null)
            return root.right;
         else if (root.right == null)
            return root.left;

         // node with two children: Get the inorder successor
(smallest
         // in the right subtree)
         root.key = minValue(root.right);

         // Delete the inorder successor
         root.right = deleteRec(root.right, root.key);
      }

      return root;
   }
```

```
        43
      /    \
    31      68
           /  \
         67    95
```

Deletion in Binary Search Tree

```
void deleteKey(int key)
   {
      root = deleteRec(root, key);
   }

   /* A recursive function to delete a key in BST */
   Node deleteRec(Node root, int key)
   {
      /* Base Case: If the bst is empty */
      if (root == null)  return root;

      /* Otherwise, recur down the bst */
      if (key < root.key)
         root.left = deleteRec(root.left, key);
      else if (key > root.key)
         root.right = deleteRec(root.right, key);

      // if key is same as root's key, then This is the node
      // to be deleted
      else
      {
         // node with only one child or no child
         if (root.left == null)
            return root.right;
         else if (root.right == null)
            return root.left;

         // node with two children: Get the inorder successor (smallest
         // in the right subtree)
         root.key = minValue(root.right);

         // Delete the inorder successor
         root.right = deleteRec(root.right, root.key);
      }

      return root;
   }
```





Deletion in Binary Search Tree

```
void deleteKey(int key)
   {
      root = deleteRec(root, key);
   }

   /* A recursive function to delete a key in BST */
   Node deleteRec(Node root, int key)
   {
      /* Base Case: If the bst is empty */
      if (root == null)  return root;

      /* Otherwise, recur down the bst */
      if (key < root.key)
         root.left = deleteRec(root.left, key);
      else if (key > root.key)
         root.right = deleteRec(root.right, key);

      // if key is same as root's key, then This is the node
      // to be deleted
      else
      {
         // node with only one child or no child
         if (root.left == null)
            return root.right;
         else if (root.right == null)
            return root.left;

         // node with two children: Get the inorder successor
(smallest
         // in the right subtree)
         root.key = minValue(root.right);

         // Delete the inorder successor
         root.right = deleteRec(root.right, root.key);
      }

      return root;
   }
```

```
        43
      /    \
    31      68
           /  \
         67    95
```

```
        67
      /    \
    31      68
              \
               95
```

Inorder traversal:
31 67 68 95